

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11 ERROR HANDLING



Disusun Oleh:
Yosafat Jacobus (105224016)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER
UNIVERSITAS PERTAMINA

2026

DAFTAR ISI

I. Pendahuluan.....	2
II. Variabel.....	3
III. Constructor dan Method.....	4
IV. Dokumentasi dan Pembahasan Code.....	7
V. Kesimpulan.....	17

I. Pendahuluan

Program JAVA EXPRESS merupakan aplikasi sederhana berbasis Java yang digunakan untuk melakukan pemesanan tiket kereta api. Program ini menerapkan konsep error handling agar kesalahan input atau kondisi tidak valid dapat ditangani dengan baik tanpa membuat program langsung berhenti secara tidak terkontrol.

Pada program ini, pengguna dapat melihat jadwal kereta, memesan tiket, dan keluar dari aplikasi. Dalam proses pemesanan, sistem melakukan beberapa validasi, seperti validasi kode kereta, validasi NIK penumpang, dan pengecekan jumlah kursi yang tersedia. Jika terdapat kesalahan, program akan menampilkan pesan error yang sesuai dengan jenis kesalahannya.

Error handling pada program ini menggunakan beberapa jenis exception. `InputMismatchException` digunakan untuk menangani kesalahan input angka, `DataPenumpangTidakValidException` digunakan untuk menangani data NIK yang tidak valid, `RuteTidakDitemukanException` digunakan saat kode kereta tidak ditemukan, dan `TiketHabisException` digunakan saat jumlah tiket yang dipesan melebihi sisa kursi. Dengan penggunaan exception tersebut, program menjadi lebih aman, rapi, dan mudah dipahami.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kodeKereta	String	Menyimpan kode unik kereta api, seperti K01 atau K02.
2	namaKereta	String	Menyimpan nama kereta api, seperti Argo Bromo atau Parahyangan.
3	rute	String	Menyimpan informasi rute perjalanan kereta, misalnya JKT - SBY.
4	sisaKursi	int	Menyimpan jumlah kursi yang masih tersedia pada kereta.
5	kapasitas	int	Menyimpan kapasitas awal kursi saat objek KeretaApi dibuat.
6	scanner	Scanner	Digunakan untuk membaca input dari pengguna melalui keyboard.
7	reservasi	SistemReservasi	Objek utama untuk menjalankan proses reservasi tiket.
8	isRunning	boolean	Menentukan apakah program masih berjalan atau sudah berhenti.
9	pilihan	int	Menyimpan pilihan menu yang dimasukkan oleh pengguna.
10	kode	String	Menyimpan kode kereta yang dimasukkan pengguna saat pemesanan.
11	nik	String	Menyimpan NIK penumpang yang akan divalidasi oleh sistem.

12	nama	String	Menyimpan nama penumpang yang melakukan pemesanan tiket.
13	jumlah	int	Menyimpan jumlah tiket yang ingin dipesan pengguna.
14	daftarKereta	List<KeretaApi>	Menyimpan daftar seluruh kereta yang tersedia dalam sistem reservasi.
15	kereta	KeretaApi	Menyimpan objek kereta hasil pencarian berdasarkan kode kereta.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	DataPenumpangTidakValidException(String pesan)	Constructor	Membuat exception untuk data penumpang yang tidak valid, khususnya NIK.
2	KeretaApi(String kodeKereta, String namaKereta, String rute, int kapasitas)	Constructor	Membuat objek kereta api dengan kode, nama, rute, dan kapasitas awal.
3	getKodeKereta()	Getter	Mengembalikan kode kereta api.
4	getNamaKereta()	Getter	Mengembalikan nama kereta api.
5	getRute()	Getter	Mengembalikan rute perjalanan kereta.

6	getSisaKursi()	Getter	Mengembalikan jumlah sisa kursi yang tersedia.
7	kurangiKursi(int jumlah)	Method	Mengurangi jumlah sisa kursi setelah pemesanan berhasil.
8	toString()	Override Method	Mengubah data objek KeretaApi menjadi format teks yang rapi.
9	main(String[] args)	Method Utama	Menjadi titik awal program dan menampilkan menu utama aplikasi.
10	prosesPemesanan(Scanner scanner, SistemReservasi reservasi)	Static Method	Mengatur input pemesanan tiket dan menangani exception yang muncul.
11	RuteTidakDitemukanException(String kodeKereta)	Constructor	Membuat checked exception saat kode kereta tidak ditemukan.
12	SistemReservasi()	Constructor	Menginisialisasi daftar kereta awal yang tersedia dalam sistem.
13	tampilkanJadwal()	Method	Menampilkan daftar kereta beserta rute dan sisa kursinya.
14	pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)	Method	Memproses pemesanan tiket setelah melewati validasi data.
15	validasiNIK(String nik)	Private Method	Memeriksa apakah NIK memiliki 16 karakter dan hanya berisi angka.

16	cariKereta(String kodeKereta)	Private Method	Mencari kereta berdasarkan kode dan melempar exception jika tidak ditemukan.
17	TiketHabisException(String namaKereta, int sisaKursi)	Constructor	Membuat checked exception saat tiket yang dipesan melebihi sisa kursi.
18	getNamaKereta()	Getter	Mengembalikan nama kereta pada exception TiketHabisException.
19	getSisaKursi()	Getter	Mengembalikan jumlah sisa kursi pada exception TiketHabisException.

IV. Dokumentasi dan Pembahasan Code

DataPenumpangTidakValidException.java

```
// Unchecked Exception → extends RuntimeException
// Dilempar saat data penumpang (NIK) tidak memenuhi validasi
public class DataPenumpangTidakValidException extends RuntimeException {
    public DataPenumpangTidakValidException(String pesan) {
        super(pesan);
    }
}
```

Class DataPenumpangTidakValidException adalah custom exception yang digunakan untuk menangani kondisi ketika data penumpang, khususnya NIK, tidak memenuhi aturan validasi. Class ini mewarisi RuntimeException, sehingga termasuk unchecked exception dan tidak wajib dideklarasikan dengan throws pada method pemanggil. Constructor menerima parameter pesan, lalu meneruskannya ke super(pesan) agar pesan error dapat ditampilkan saat exception ditangkap. Dalam program ini, exception tersebut dilempar oleh method validasiNIK() di class SistemReservasi ketika NIK tidak berjumlah tepat 16 karakter atau mengandung karakter selain angka.

RuteTidakDitemukanException.java

```
// Checked Exception → extends Exception
// Dilempar saat kode kereta yang dicari tidak ada dalam sistem
public class RuteTidakDitemukanException extends Exception {
    public RuteTidakDitemukanException(String kodeKereta) {
        super("Rute dengan kode kereta " + kodeKereta + " tidak ditemukan dalam sistem.");
    }
}
```

Class RuteTidakDitemukanException adalah custom exception yang digunakan untuk menangani kondisi ketika kode kereta yang dimasukkan pengguna tidak ditemukan dalam daftar kereta. Class ini mewarisi Exception, sehingga termasuk checked exception dan wajib dideklarasikan dengan throws serta ditangani menggunakan try-catch. Constructor menerima parameter kodeKereta, lalu mengirim pesan error ke super(...) agar pesan tersebut dapat ditampilkan saat exception ditangkap. Dalam program ini, exception tersebut dilempar oleh method cariKereta() di class SistemReservasi ketika tidak ada kereta yang memiliki kode sesuai input pengguna.

TiketHabisException.java

```
// Checked Exception → extends Exception
// Dilempar saat jumlah tiket yang dipesan melebihi sisa kursi
// Membawa informasi spesifik: nama kereta dan sisa kursi yang tersedia
public class TiketHabisException extends Exception {
    private String namaKereta;
    private int sisaKursi;

    public TiketHabisException(String namaKereta, int sisaKursi) {
        // Pesan error otomatis dibentuk dari nama kereta dan sisa kursi
        super("Tiket untuk kereta " + namaKereta + " tidak mencukupi. "
            + "Sisa kursi tersedia: " + sisaKursi);
        this.namaKereta = namaKereta;
        this.sisaKursi = sisaKursi;
    }

    // Getter untuk mengakses info spesifik dari exception ini
    public String getNamaKereta() { return namaKereta; }
    public int getSisaKursi() { return sisaKursi; }
}
```

Class TiketHabisException adalah custom exception yang digunakan untuk menangani kondisi ketika jumlah tiket yang dipesan melebihi sisa kursi yang tersedia. Class ini mewarisi Exception, sehingga termasuk checked exception dan wajib dideklarasikan dengan throws serta ditangani menggunakan try-catch. Constructor menerima parameter namaKereta dan sisaKursi, lalu membentuk pesan error melalui super(...) agar informasi kereta dan jumlah kursi yang tersisa dapat ditampilkan. Dalam program ini, exception tersebut dilempar oleh method pesanTiket() di class SistemReservasi ketika jumlahTiket lebih besar dari kereta.getSisaKursi(). Getter getNamaKereta() dan getSisaKursi() disediakan agar informasi spesifik dari exception, yaitu nama kereta dan sisa kursi, dapat diakses secara terpisah oleh blok catch yang menangkapnya.

KeretaApi.java

```
// Kelas model yang merepresentasikan satu objek kereta api
public class KeretaApi {
    private String kodeKereta;
    private String namaKereta;
    private String rute;
    private int sisaKursi;
```

```

// Konstruktor menerima kapasitas awal sebagai sisa kursi
public KeretaApi(String kodeKereta, String namaKereta, String rute, int kapasitas) {
    this.kodeKereta = kodeKereta;
    this.namaKereta = namaKereta;
    this.rute = rute;
    this.sisaKursi = kapasitas;
}

// Getter untuk mengakses data kereta
public String getKodeKereta() { return kodeKereta; }
public String getNamaKereta() { return namaKereta; }
public String getRute() { return rute; }
public int getSisaKursi() { return sisaKursi; }

// Mengurangi sisa kursi setelah pemesanan berhasil
// Validasi jumlah sudah dilakukan di SistemReservasi sebelum method ini dipanggil
public void kurangiKursi(int jumlah) {
    this.sisaKursi -= jumlah;
}

// Representasi teks objek kereta untuk ditampilkan di jadwal
@Override
public String toString() {
    return String.format("%-6s %-20s %-12s %d kursi",
        kodeKereta, namaKereta, rute, sisaKursi);
}
}

```

Class KeretaApi adalah class model yang digunakan untuk merepresentasikan satu data kereta api dalam sistem reservasi. Class ini memiliki atribut kodeKereta, namaKereta, rute, dan sisaKursi yang dibuat private agar data hanya dapat diakses melalui method yang disediakan. Constructor KeretaApi() digunakan untuk mengisi data awal kereta, dengan parameter kapasitas yang langsung disimpan sebagai sisaKursi. Method getter digunakan untuk mengambil informasi kereta tanpa mengubah datanya. Method kurangiKursi() digunakan untuk mengurangi sisa kursi setelah pemesanan berhasil, tanpa melakukan validasi di dalamnya karena pengecekan jumlah kursi sudah dilakukan sebelumnya oleh SistemReservasi. Selain itu, method toString() dioverride agar objek kereta dapat ditampilkan dalam format teks yang lebih rapi pada jadwal.

SistemReservasi.java

```
import java.util.ArrayList;
import java.util.List;

// Kelas pusat kontrol reservasi tiket kereta api
// Bertanggung jawab atas validasi, pencarian kereta, dan proses pemesanan
public class SistemReservasi {

    private List<KeretaApi> daftarKereta; // menyimpan semua data kereta yang tersedia

    public SistemReservasi() {
        daftarKereta = new ArrayList<>();
        // Data awal kereta diinisialisasi saat sistem pertama kali dijalankan
        daftarKereta.add(new KeretaApi("K01", "Argo Bromo", "JKT - SBY", 50));
        daftarKereta.add(new KeretaApi("K02", "Parahyangan", "JKT - BDG", 15));
    }

    // Menampilkan seluruh jadwal kereta yang tersedia beserta sisa kursinya
    public void tampilkanJadwal() {
        System.out.println("\nJADWAL KERETA JAVA EXPRESS");
        System.out.printf("%-6s %-20s %-12s %s%n",
            "Kode", "Nama Kereta", "Rute", "Sisa Kursi");
        for (KeretaApi k : daftarKereta) {
            System.out.printf("%-6s %-20s %-12s %d kursi%n",
                k.getKodeKereta(),
                k.getNamaKereta(),
                k.getRute(),
                k.getSisaKursi());
        }
        System.out.println();
    }
}
```

Class SistemReservasi adalah class yang berperan sebagai pusat kontrol dalam sistem pemesanan tiket kereta api. Class ini bertanggung jawab untuk menyimpan daftar kereta, menampilkan jadwal, melakukan validasi, mencari data kereta, dan memproses pemesanan tiket. Pada bagian awal, program mengimpor ArrayList dan List karena data kereta disimpan dalam bentuk collection.

Atribut daftarKereta bertipe List<KeretaApi> digunakan untuk menyimpan seluruh objek kereta yang tersedia. Pada constructor SistemReservasi(), list tersebut diinisialisasi menggunakan ArrayList, lalu diisi dengan data awal kereta, yaitu K01 Argo Bromo dengan rute JKT - SBY dan K02 Parahyangan dengan rute JKT - BDG.

Method `tampilkanJadwal()` digunakan untuk menampilkan daftar kereta yang tersedia kepada pengguna. Data yang ditampilkan meliputi kode kereta, nama kereta, rute, dan sisa kursi. Perulangan `for-each` digunakan untuk mengambil setiap objek `KeretaApi` dari `daftarKereta`, kemudian data kereta ditampilkan melalui getter seperti `getKodeKereta()`, `getNamaKereta()`, `getRute()`, dan `getSisaKursi()`.

```
// Metode utama pemesanan tiket
// Checked Exception dideklarasikan eksplisit di signature method
public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
    throws RuteTidakDitemukanException, TiketHabisException {

    // Langkah 1: validasi NIK penumpang → melempar Unchecked Exception jika tidak valid
    validasiNIK(nik);

    // Langkah 2: cari kereta berdasarkan kode → melempar Checked Exception jika tidak ditemukan
    KeretaApi kereta = cariKereta(kodeKereta);

    // Langkah 3: cek ketersediaan kursi → melempar Checked Exception jika tidak cukup
    if (jumlahTiket > kereta.getSisaKursi()) {
        throw new TiketHabisException(kereta.getNamaKereta(), kereta.getSisaKursi());
    }

    // Langkah 4: semua validasi lolos, proses pemesanan dijalankan
    kereta.kurangiKursi(jumlahTiket);
    System.out.println("\nPemesanan berhasil!");
    System.out.printf("Penumpang : %s%n", namaPenumpang);
    System.out.printf("NIK : %s%n", nik);
    System.out.printf("Kereta : %s%n", kereta.getNamaKereta());
    System.out.printf("Rute : %s%n", kereta.getRute());
    System.out.printf("Jumlah : %d tiket%n", jumlahTiket);
    System.out.printf("Sisa Kursi : %d kursi%n", kereta.getSisaKursi());
    System.out.println();
}

// Memvalidasi format NIK: harus tepat 16 karakter dan hanya berisi angka
// Melempar DataPenumpangTidakValidException (Unchecked) jika tidak sesuai
private void validasiNIK(String nik) {
    if (nik.length() != 16) {
        throw new DataPenumpangTidakValidException(
            "NIK tidak valid! Harus tepat 16 karakter. "
            + "NIK Anda memiliki " + nik.length() + " karakter.");
    }
}
```

```

        if (!nik.matches("\\d+")) {
            throw new DataPenumpangTidakValidException(
                "NIK tidak valid! NIK hanya boleh mengandung angka.");
        }
    }

    // Mencari objek KeretaApi berdasarkan kode kereta (case-insensitive)
    // Melempar RuteTidakDitemukanException (Checked) jika kode tidak ditemukan
    private KeretaApi cariKereta(String kodeKereta) throws RuteTidakDitemukanException {
        for (KeretaApi k : daftarKereta) {
            if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)) {
                return k;
            }
        }
        // Hanya pass kodeKereta, bukan kalimat lengkap — pesan dibentuk di konstruktor exception
        throw new RuteTidakDitemukanException(kodeKereta);
    }
}

```

Method `pesanTiket()` merupakan method utama dalam proses pemesanan tiket. Method ini menerima input berupa `kodeKereta`, `nik`, `namaPenumpang`, dan `jumlahTiket`. Pada signature method terdapat `throws` `RuteTidakDitemukanException`, `TiketHabisException` karena method ini dapat melempar checked exception ketika kode kereta tidak ditemukan atau jumlah tiket melebihi sisa kursi.

Proses pemesanan dilakukan secara bertahap. Pertama, method `validasiNIK(nik)` dipanggil untuk memastikan NIK memiliki tepat 16 karakter dan hanya berisi angka. Jika NIK tidak valid, method tersebut akan melempar `DataPenumpangTidakValidException`, yaitu unchecked exception. Kedua, method `cariKereta(kodeKereta)` digunakan untuk mencari objek kereta berdasarkan kode yang dimasukkan pengguna. Jika kode tidak ditemukan, method ini melempar `RuteTidakDitemukanException`.

Setelah kereta ditemukan, program mengecek apakah `jumlahTiket` lebih besar dari `kereta.getSisaKursi()`. Jika jumlah tiket yang diminta melebihi kursi yang tersedia, program akan melempar `TiketHabisException` dengan membawa informasi nama kereta dan sisa kursi. Jika seluruh validasi berhasil, method `kurangiKursi(jumlahTiket)` dipanggil untuk mengurangi sisa kursi, lalu program menampilkan detail pemesanan berhasil.

Method `validasiNIK()` dibuat private karena hanya digunakan di dalam class `SistemReservasi`. Method ini memvalidasi NIK berdasarkan dua syarat, yaitu panjang NIK harus tepat 16 karakter dan seluruh karakter harus berupa angka. Sementara itu, method `cariKereta()` juga dibuat private karena hanya berfungsi sebagai proses internal untuk mencari kereta di dalam `daftarKereta`. Pencarian dilakukan menggunakan perulangan `for-each` dan `equalsIgnoreCase()` agar kode

kereta tetap dapat dikenali meskipun pengguna memasukkan huruf besar atau kecil.

Main.java

```
import java.util.InputMismatchException;
import java.util.Scanner;

// Kelas utama program — titik masuk aplikasi JAVA EXPRESS
public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        SistemReservasi reservasi = new SistemReservasi();
        boolean isRunning = true;

        System.out.println("Selamat datang di JAVA EXPRESS");
        System.out.println("Sistem Pemesanan Tiket Kereta Api");

        try {
            while (isRunning) {
                // Tampilkan menu utama setiap iterasi
                System.out.println("\nMenu Utama");
                System.out.println("1. Lihat Jadwal Kereta");
                System.out.println("2. Pesan Tiket");
                System.out.println("3. Keluar");
                System.out.print("Pilih menu: ");

                try {
                    int pilihan = scanner.nextInt();
                    scanner.nextLine(); // bersihkan newline setelah nextInt()

                    switch (pilihan) {
                        case 1:
                            reservasi.tampilkanJadwal();
                            break;
                        case 2:
                            prosesPemesanan(scanner, reservasi);
                            break;
                        case 3:
                            isRunning = false;
                            break;
                    }
                }
            }
        }
    }
}
```

```

        default:
            System.out.println("Pilihan tidak valid. Masukkan angka 1, 2, atau 3.");
        }

    } catch (InputMismatchException e) {
        // Pengguna memasukkan huruf/simbol saat memilih menu
        System.out.println("Input tidak valid! Harap masukkan angka.");
        scanner.nextLine(); // WAJIB: bersihkan buffer agar tidak terjebak infinite loop
    }
}

} finally {
    // Blok ini dijamin selalu tereksekusi saat program keluar,
    // baik karena pilihan user (case 3) maupun karena exception tak tertangkap
    System.out.println("\nTerima kasih telah menggunakan JAVA EXPRESS.");
    System.out.println("Sampai jumpa!");
    scanner.close(); // tutup Scanner untuk melepas resource
}
}

```

Class Main merupakan class utama yang menjadi titik awal jalannya program JAVA EXPRESS. Pada method main(), program membuat objek Scanner untuk membaca input pengguna dan objek SistemReservasi untuk mengakses fitur reservasi tiket. Variabel isRunning digunakan untuk mengontrol perulangan menu utama agar program tetap berjalan sampai pengguna memilih keluar.

Di dalam perulangan while, program menampilkan tiga pilihan menu, yaitu melihat jadwal kereta, memesan tiket, dan keluar dari aplikasi. Input menu dibaca menggunakan scanner.nextInt(), kemudian scanner.nextLine() dipanggil untuk membersihkan karakter newline yang tersisa di buffer. Setelah itu, struktur switch digunakan untuk menentukan aksi berdasarkan pilihan pengguna. Jika pengguna memilih menu 1, program memanggil tampilkanJadwal(). Jika memilih menu 2, program memanggil prosesPemesanan(). Jika memilih menu 3, nilai isRunning diubah menjadi false agar perulangan berhenti.

Program juga menggunakan try-catch untuk menangani InputMismatchException, yaitu kesalahan yang terjadi ketika pengguna memasukkan huruf atau simbol saat program meminta input angka. Pada blok catch, program menampilkan pesan error dan memanggil scanner.nextLine() untuk membersihkan buffer agar program tidak terjebak dalam infinite loop.

Blok finally digunakan untuk memastikan pesan penutup tetap ditampilkan dan scanner.close() tetap dijalankan setelah loop selesai, baik karena pengguna memilih keluar maupun karena terjadi exception yang tidak tertangkap di dalam proses perulangan. Dengan struktur ini, program menjadi lebih aman, rapi, dan mampu menangani kesalahan input pengguna dengan baik.

```

// Menangani seluruh alur input dan proses pemesanan tiket
// Semua exception dari SistemReservasi ditangkap di sini
private static void prosesPemesanan(Scanner scanner, SistemReservasi reservasi) {
    System.out.println("\nForm Pemesanan Tiket");

    try {
        System.out.print("Kode Kereta   : ");
        String kode = scanner.nextLine().trim().toUpperCase(); // normalisasi ke huruf kapital

        System.out.print("NIK Penumpang : ");
        String nik = scanner.nextLine().trim();

        System.out.print("Nama Penumpang : ");
        String nama = scanner.nextLine().trim();

        System.out.print("Jumlah Tiket   : ");
        int jumlah = scanner.nextInt();
        scanner.nextLine(); // bersihkan buffer setelah nextInt()

        // Serahkan ke SistemReservasi untuk diproses
        reservasi.pesanTiket(kode, nik, nama, jumlah);

    } catch (InputMismatchException e) {
        // Pengguna memasukkan huruf saat mengisi jumlah tiket
        System.out.println("Jumlah tiket harus berupa angka!");
        scanner.nextLine(); // bersihkan buffer
    }

    } catch (DataPenumpangTidakValidException e) {
        // Unchecked Exception: NIK tidak sesuai format
        System.out.println("Data tidak valid: " + e.getMessage());
    }

    } catch (RuteTidakDitemukanException e) {
        // Checked Exception: kode kereta tidak ada dalam sistem
        System.out.println("Rute tidak ditemukan: " + e.getMessage());
    }

    } catch (TiketHabisException e) {
        // Checked Exception: kursi tidak cukup untuk jumlah yang dipesan
        System.out.println("Tiket tidak mencukupi: " + e.getMessage());
        System.out.println("Coba kurangi jumlah tiket atau pilih kereta lain.");
    }
}
}

```


Method prosesPemesanan() digunakan untuk menangani seluruh alur input pemesanan tiket dari pengguna. Method ini dibuat private static karena hanya dipakai di dalam class Main dan dapat dipanggil langsung dari method main() yang juga bersifat static. Parameter Scanner scanner digunakan untuk membaca input pengguna, sedangkan SistemReservasi reservasi digunakan untuk meneruskan data pemesanan ke sistem reservasi.

Di dalam method ini, pengguna diminta memasukkan kode kereta, NIK penumpang, nama penumpang, dan jumlah tiket. Input kode kereta diproses dengan trim().toUpperCase() agar spasi berlebih dihapus dan kode kereta diseragamkan menjadi huruf kapital. Setelah seluruh data diisi, method reservasi.pesanTiket(kode, nik, nama, jumlah) dipanggil untuk menjalankan validasi dan proses pemesanan.

Method ini juga memiliki beberapa blok catch untuk menangani error yang mungkin terjadi. InputMismatchException menangani kondisi ketika jumlah tiket diisi bukan angka. DataPenumpangTidakValidException menangani NIK yang tidak sesuai format. RuteTidakDitemukanException menangani kode kereta yang tidak tersedia. Sementara itu, TiketHabisException menangani kondisi ketika jumlah tiket yang dipesan melebihi sisa kursi. Dengan struktur ini, setiap kesalahan dapat ditangani secara spesifik dan pengguna mendapatkan pesan error yang jelas.

V. Kesimpulan

Berdasarkan program JAVA EXPRESS yang telah dibuat, dapat disimpulkan bahwa error handling berperan penting dalam menjaga program tetap berjalan dengan aman ketika terjadi kesalahan input atau kondisi tidak valid. Sesuai materi Modul 11, exception handling digunakan agar program tidak langsung berhenti secara tiba-tiba, tetapi dapat menampilkan pesan kesalahan yang lebih jelas kepada pengguna.

Program ini telah menerapkan konsep try-catch-finally dengan baik. Blok try digunakan untuk menjalankan kode yang berpotensi menimbulkan error, sedangkan blok catch digunakan untuk menangani error secara spesifik, seperti kesalahan input angka, NIK tidak valid, rute tidak ditemukan, dan tiket yang tidak mencukupi. Blok finally digunakan untuk memastikan resource seperti Scanner tetap ditutup setelah program selesai digunakan.

Selain itu, program ini juga menerapkan custom exception sesuai kebutuhan sistem reservasi tiket. `DataPenumpangTidakValidException` digunakan sebagai unchecked exception untuk menangani NIK yang tidak sesuai format. Sementara itu, `RuteTidakDitemukanException` dan `TiketHabisException` digunakan sebagai checked exception karena kondisi tersebut wajib ditangani secara eksplisit oleh program menggunakan throws dan try-catch.

Penggunaan throw dan throws juga terlihat jelas dalam program. Keyword throw digunakan untuk melempar exception ketika suatu kondisi tidak terpenuhi, seperti NIK salah, kode kereta tidak ditemukan, atau jumlah tiket melebihi sisa kursi. Keyword throws digunakan pada method yang berpotensi menghasilkan checked exception, sehingga tanggung jawab penanganan error menjadi lebih jelas.

Secara keseluruhan, program JAVA EXPRESS sudah menerapkan konsep error handling pada Modul 11 secara tepat. Program menjadi lebih terstruktur, mudah dipahami, dan mampu menangani kesalahan pengguna dengan pesan yang informatif. Dengan adanya pembagian class, validasi data, custom exception, serta pembersihan buffer input menggunakan `scanner.nextLine()`, program menjadi lebih stabil dan sesuai dengan prinsip pemrograman berorientasi objek.